

RentalApp

Peer-to-Peer Library of Things

SET09102 - Software Engineering

Report detail	Value
Student	Justin Wylie
Matriculation number	40535392
Target	70% band: complete Tier 1 and Tier 2 plus advanced State Pattern
Submission date	26 July 2026
Public GitHub URL	ADD PUBLIC URL BEFORE SUBMISSION

Evidence status

All source-based sections are populated and one real Near me screenshot is included. Before submission, add the public repository URL, seven remaining emulator captures, the coverage-report screenshot, and a green GitHub Actions run.

1. Project overview

RentalApp is a compact Android marketplace in which community members list useful items, find nearby listings, request rentals, progress a controlled rental workflow, and leave verified reviews. The implementation deliberately completes the brief's Tier 1 and Tier 2 requirements and adds the State Pattern and automatic overdue detection from Tier 3.

Submission claim

Evidence-based position

The source is shaped for the 70% band, but a grade is not guaranteed by feature count. The presentation is worth 50% of the module and requires the student to explain and defend the design. The final verified 86.4% line coverage exceeds the brief's 80% distinction testing threshold.

Technology stack

Area	Technology	Evidence
Client	.NET 10 / .NET MAUI	Android XAML, Shell, Secure Storage, GPS, address geocoding
API	ASP.NET Core 10	Minimal API endpoints, JWT bearer authentication, Problem Details
Data	EF Core 10	DbContext, migrations, repositories, Unit of Work
Spatial	PostgreSQL 16 + PostGIS	Geography point, SRID 4326, GiST, radius query
Quality	xUnit + Coverlet	65/65 passing tests; 86.4% line coverage
DevOps	Docker Compose + GitHub Actions	Healthy local services, test/coverage job, signed APK job
Environment	VS Code Dev Container	.NET 10 Noble image and C# / Docker extensions

Report map

Architecture 3-6 | Features 7-9 | Testing 10-11 | CI/CD 12 | Patterns 13-14 | AI use 15 |
References and final gates 16

2. Architecture documentation

2.1 Component diagram

RentalApp component architecture

Dependencies flow through interfaces; business rules remain at the API boundary.

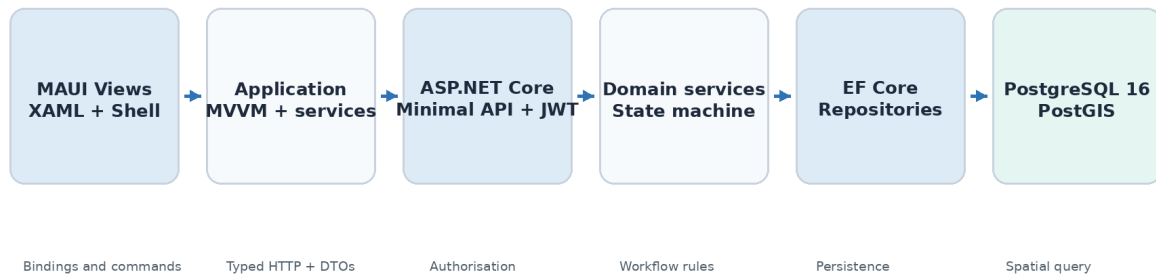


Figure 1. Component boundaries and dependency direction.

The mobile client never connects directly to PostgreSQL. Views bind to ViewModels, application services call the authenticated HTTP client, and the API owns security and business rules. EF Core repositories isolate persistence and spatial queries. This supports testability and a clear separation of concerns.

Design rationale

Interfaces keep Android concerns out of ViewModel tests, while the API remains the authoritative boundary even if a client bypasses the UI.

2.2 Database schema

Database schema

UUID keys, explicit relationships, spatial indexing, and review integrity.

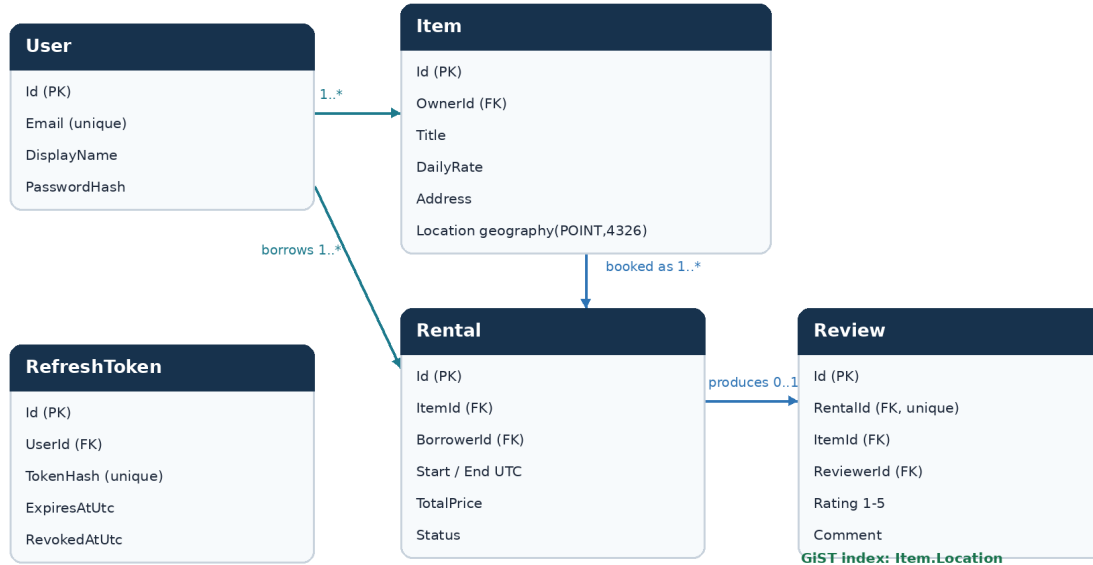


Figure 2. Core entities and integrity relationships.

Item.Address stores the readable collection address; the MAUI client forward-geocodes typed addresses or reverse-geocodes device GPS. Item.Location is mapped to geography (point, 4326), giving distance in metres. A GiST index supports the nearby predicate. Reviews have a unique RentalId and rating check constraint.

2.3 Rental request sequence

Rental request sequence

The API authenticates the caller and enforces booking rules before persistence.

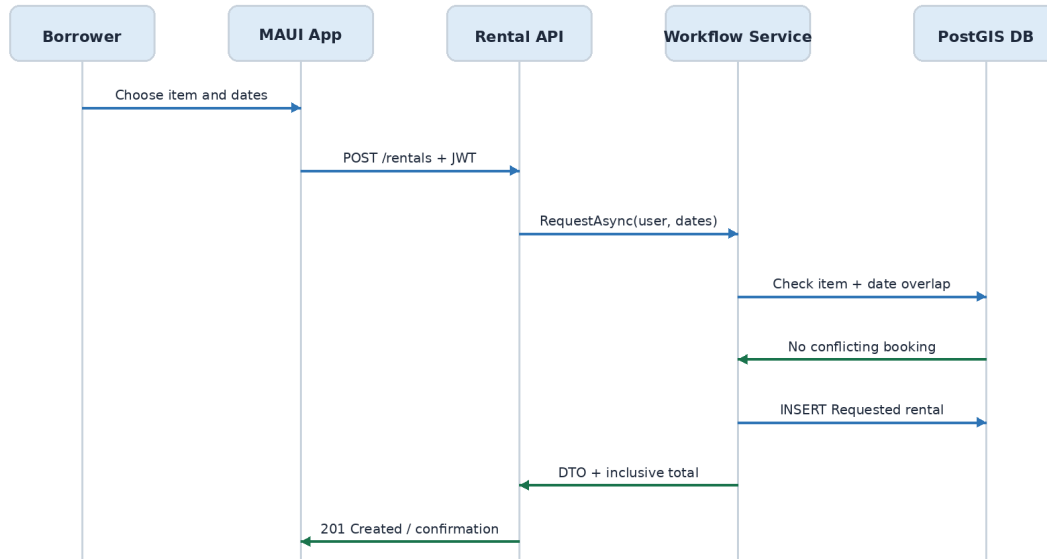


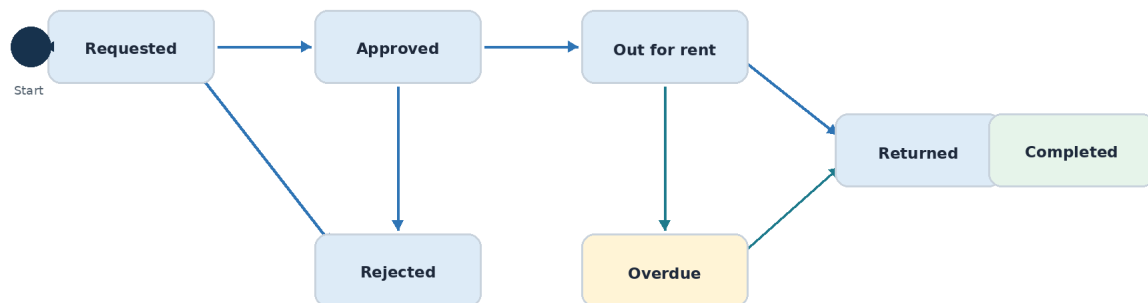
Figure 3. Successful rental request from the MAUI app to PostgreSQL.

RentalWorkflowService validates future dates, availability, self-rental, and inclusive overlap before calculating price and committing. The response includes the Requested status and total price. Exceptions are converted to consistent API errors and displayed by the ViewModel.

2.4 Rental state diagram

Rental state machine

State classes validate workflow transitions; the service validates the acting user's role.



Automatic transition: Out for rent -> Overdue when the due date passes.

Figure 4. Permitted workflow transitions implemented by state classes.

Each state declares its permitted successors. Role authorisation is intentionally separate: owners approve, reject, start, and complete; borrowers return. The background overdue worker checks expired out-for-rent records and applies the same state machine.

3. Feature implementation checklist

Done	Requirement	Implementation evidence
YES	Registration and sign-in	Password hashing, JWT access token, rotating hashed refresh token
YES	Authenticated client	Bearer injection, refresh flow, Secure Storage, sign-out
YES	Item catalogue	Address entry/geocoding, create, browse, detail, owner edit, availability
YES	Nearby discovery	Address or GPS position, category/radius filters, distance result
YES	PostGIS	Geography point, GIST index, IsWithinDistance query
YES	Rental request	Future dates, inclusive price, self-rental and overlap prevention
YES	Rental workflow	Incoming/outgoing lists and role-aware actions
YES	Verified reviews	Borrower only, completed rental only, one review per rental
YES	MVVM	Observable ViewModels, RelayCommand, binding-first XAML
YES	Repository / Unit of Work	Generic plus item, rental, and review repositories
YES	Service Layer	Client services and API business services
YES	State Pattern	Seven state classes with parameterised transition tests
YES	Overdue detection	Hosted worker moves expired records to Overdue
YES	Testing	65/65 tests and 86.4% line coverage, including HTTP/PostGIS integration
YES	CI/CD source	PR/push pipeline, PostGIS service, coverage artifact, signed APK
YES	Docker / Dev Container	Compose health checks and .NET 10 Noble environment
YES	Code documentation	XML summaries on key public contracts and 29 presentation-point comments
NO	MediatR / CQRS Lite	Optional; excluded to keep the system compact and explainable
NO	SonarCloud	Optional; coverage artifacts are implemented instead

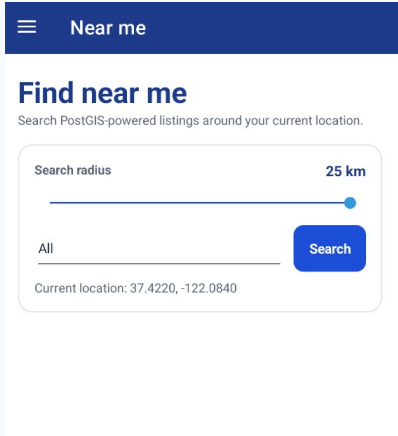
Scope decision: the brief makes MediatR and SonarCloud optional. The implementation invests in a fully explained State Pattern, PostGIS integration test, mobile workflow, and reproducible deployment instead of adding optional infrastructure without evidence.

3.1 Feature screenshots - authentication and catalogue

Replace each marked evidence slot with a cropped emulator screenshot. Keep the caption and ensure user data contains no private credentials.

A. Login / registration INSERT RUNNING-APP SCREENSHOT <i>Show both actions and validation feedback.</i>	B. Browse items INSERT RUNNING-APP SCREENSHOT <i>Show seeded catalogue and category filter.</i>
C. Item detail / request INSERT RUNNING-APP SCREENSHOT <i>Show rate, owner, reviews, and date selection.</i>	D. Owner edit INSERT RUNNING-APP SCREENSHOT <i>Show owner-only fields and availability control.</i>

3.2 Feature screenshots - spatial, workflow, and review

E. Create item INSERT RUNNING-APP SCREENSHOT <i>Show category, daily rate, address lookup, and confirmation.</i>	F. Near me  <i>Real emulator capture showing radius/category controls and device coordinates.</i>
G. Rental workflow INSERT RUNNING-APP SCREENSHOT <i>Show incoming/outgoing status and valid action buttons.</i>	H. Verified review / profile INSERT RUNNING-APP SCREENSHOT <i>Show completed-rental picker and profile summary.</i>
High-value demonstration Use Mike to request, Sarah to approve/start, Mike to return/review, and Sarah to complete. This proves role-based actions and the state workflow rather than only static screens.	

4. Testing documentation

4.1 Verified result and coverage

Verified Docker test run - 26 July 2026

65 total | 65 passed | 0 failed | 0 skipped | 16 seconds. Cobertura line coverage: 86.4%.

Distinction testing threshold met

The final build's measured 86.4% line rate exceeds the brief's 80% distinction threshold. Insert a screenshot/export of the Cobertura or generated HTML report as visual evidence.

Test group	Cases	Primary concern
RentalApiTests	3	Full authenticated HTTP workflows against PostGIS
ItemRepositoryTests	1	Real PostgreSQL/PostGIS radius integration
API client + client services	11	HTTP, refresh, auth, item, rental, and review routes
API application services	19	Validation, ownership, price, overlap, review, JWT, overdue
RentalStateMachineTests	11	7 valid + 4 invalid transition cases
ViewModel tests (8 classes)	20	Busy/error state, role-aware commands, navigation, reviews, profile

Total: 65 cases across 19 test classes. Theory and InlineData cases are counted as executed xUnit cases.

4.2 Representative test excerpts

Example 1 - parameterised State Pattern test

```
[Theory]
[InlineData(RentalStatus.Requested, RentalStatus.Approved)]
[InlineData(RentalStatus.OutForRent, RentalStatus.Overdue)]
[InlineData(RentalStatus.Returned, RentalStatus.Completed)]
public void EnsureValidTransition_AllowedTransition_DoesNotThrow(
    RentalStatus current, RentalStatus next)
{
    var exception = Record.Exception(
        () => _machine.EnsureValidTransition(current, next));
    Assert.Null(exception);
}
```

Example 2 - business rule test

```
[Fact]
public async Task RequestAsync_OverlappingDates_RejectsSecondRequest()
{
    var data = await SeedUsersAndItemAsync(context);
    var start = DateTimeOffset.UtcNow.Date.AddDays(4);
    await service.RequestAsync(data.Borrower.Id,
        new(data.Item.Id, start, start.AddDays(2)));

    await Assert.ThrowsAsync<BusinessRuleException>(() =>
        service.RequestAsync(data.SecondBorrower.Id,
            new(data.Item.Id, start.AddDays(1), start.AddDays(3))));
}
```

Example 3 - real PostGIS integration test

```
[Fact]
public async Task GetNearbyAsync_ItemsInsideAndOutsideRadius_ReturnsOnlyNearbyItem()
{
    var results = await repository.GetNearbyAsync(
        55.9533, -3.1883, 2, ItemCategory.Tools);

    Assert.Contains(results, x => x.Item.Id == nearby.Id);
    Assert.DoesNotContain(results, x => x.Item.Id == distant.Id);
    Assert.All(results, x => Assert.InRange(x.DistanceMetres, 0, 2_000));
}
```

The tests follow Arrange-Act-Assert, use descriptive names, and combine fast in-memory/mocked tests with one real spatial database fixture where an in-memory substitute would not prove translation to PostGIS.

5. CI/CD evidence

The workflow runs on pushes and pull requests to main. The backend job starts PostgreSQL 16 with PostGIS, restores, builds, executes tests, creates HTML/Cobertura/text coverage, and uploads evidence. A separate Android job installs Java and the MAUI workload, creates an ephemeral coursework signing key, publishes a signed Release APK, and uploads it.

`.github/workflows/build.yml` (abridged)

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  backend-tests:
    services:
      postgres:
        image: postgis/postgis:16-3.5-alpine
    steps:
      - run: dotnet test RentalApp.Test/RentalApp.Test.csproj
        -c Release --no-build --collect:"XPlat Code Coverage"
      - uses: actions/upload-artifact@v4

  android-build:
    steps:
      - run: dotnet workload install maui-android
      - run: dotnet publish RentalApp/RentalApp.csproj
        -c Release -f net10.0-android
      - uses: actions/upload-artifact@v4
```

INSERT GREEN WORKFLOW SCREENSHOT

Capture the GitHub Actions run showing both backend-tests and android-build green. Also download the test-results-and-coverage and rentalapp-android artifacts. A workflow file alone earns only half of the CI/CD report evidence marks.

6. Design patterns implementation

6.1 MVVM

Views define layout and bind to observable state. ViewModels expose commands and depend on abstractions, so catalogue behaviour can be tested without a page or emulator. Generated ObservableProperty and RelayCommand members reduce repetitive notification code.

MVVM example - ItemsListViewModel

```
public partial class ItemsListViewModel(
    ItemService items, INavigationService navigation) : ViewModelBase
{
    public ObservableCollection<ItemSummaryDto> Items { get; } = [];

    [RelayCommand]
    private Task LoadAsync() => RunBusyAsync(async () =>
    {
        var results = await items.GetAllAsync(category);
        Items.Clear();
        foreach (var item in results) Items.Add(item);
    });
}
```

6.2 Repository and Unit of Work

Repositories centralise EF Core expressions and eager-loading choices. The Item repository is the only layer that knows the spatial predicate. Unit of Work provides one explicit SaveChanges boundary for service operations.

Repository example - PostGIS radius query

```
var origin = new Point(longitude, latitude) { SRID = 4326 };
var radiusMetres = radiusKilometres * 1_000;

var results = await Context.Items
    .AsNoTracking()
    .Where(item => item.IsAvailable &&
        item.Location.IsWithinDistance(origin, radiusMetres))
    .Select(item => new {
        Item = item,
        DistanceMetres = item.Location.Distance(origin)
    })
    .OrderBy(result => result.DistanceMetres)
    .ToListAsync(cancellationToken);
```

6.3 Service Layer

RentalWorkflowService is the authoritative use-case layer. It coordinates repositories, validates rules, calculates inclusive price, checks actor permissions, invokes the state machine, and commits. Endpoints remain thin and ViewModels cannot bypass these rules.

Service example - validation and inclusive pricing

```
if (item.OwnerId == borrowerId)
    throw new BusinessException("You cannot rent your own item.");

if (await rentals.HasDateOverlapAsync(item.Id, start, end, cancellationTokens))
    throw new BusinessException("The item is already booked for those dates.");

var numberOfDays = (end.Date - start.Date).Days + 1;
var rental = new Rental {
    ItemId = item.Id,
    BorrowerId = borrowerId,
    StartDateUtc = start,
    EndDateUtc = end,
    TotalPrice = item.DailyRate * numberOfDays
};
```

6.4 State Pattern (advanced)

A state object owns the allowed outgoing transitions for one RentalStatus. Adding a state or transition is localised instead of extending a large conditional. RentalWorkflowService separately checks roles, preserving single responsibility.

Advanced pattern example - explicit state classes

```
public abstract class RentalState : IRentalState
{
    private readonly HashSet<RentalStatus> _allowed;

    protected RentalState(params RentalStatus[] allowed)
    {
        _allowed = new HashSet<RentalStatus>(allowed);
    }

    public abstract RentalStatus Status { get; }
    public bool CanTransitionTo(RentalStatus next)
    {
        return _allowed.Contains(next);
    }
}

public sealed class OutForRentState : RentalState
{
    public OutForRentState()
        : base(RentalStatus.Overdue, RentalStatus.Returned) { }

    public override RentalStatus Status => RentalStatus.OutForRent;
}
```

7. AI tool usage

Tools used: ChatGPT Codex for architecture, implementation support, refactoring, test design, debugging, and documentation review; Visual Studio Code C# Dev Kit for compiler diagnostics, navigation, and debugging.

Interaction	Suggestion	Evaluation / modification	Validation
Architecture	Separate MAUI, application, API, data, migrations, and test projects.	Accepted for testability and security. Rejected optional MediatR/SonarCloud to control scope.	Build, dependency, service, and integration tests.
Spatial search	Use SRID 4326 Point, geography column, GiST, and IsWithinDistance.	Checked X=longitude, Y=latitude; validated coordinate/radius bounds.	Real PostGIS fixture returns near item only.
Rental workflow	One class per state; keep role checks in the service.	Accepted to demonstrate Open/Closed design; overlap and inclusive-price rules retained.	Theory tests plus price, overlap, role, and overdue tests.
Android debugging	Use Logcat to diagnose splash exit and details crash.	Changes tied to observed stack traces: embed assemblies, correct App initialisation, harden route.	Clean rebuild/install and focused ViewModel tests.
Address entry	Forward-geocode typed addresses and reverse-geocode GPS.	Store readable address plus validated PostGIS point; remove raw coordinate fields from UI.	ViewModel and authenticated API workflow tests.
Unified accounts and review UX	Derive actions from listing-creator/requester relationships and carry review counts through DTOs.	Kept every API ownership and requester check; the UI is helpful but is not the security boundary.	Final regression: 65/65 tests and 86.4% coverage.

Reflection

AI accelerated repetitive implementation and exposed alternatives, but it did not replace engineering judgement. Suggestions were narrowed to an explainable scope, compiled with warnings as errors, and tested at the appropriate boundary. Database and Android defects were diagnosed from evidence instead of accepting speculative changes. In the presentation, the student must explain every excerpt above and the trade-offs behind it.

8. References

1. [Microsoft \(2026\), .NET MAUI documentation](#)
2. [Microsoft \(2025\), Spatial data in Entity Framework Core](#)
3. [Microsoft \(2025\), Configure JWT bearer authentication in ASP.NET Core](#)
4. [PostGIS Project, ST_DWithin reference](#)
5. [NetTopologySuite project, API documentation](#)
6. [xUnit.net, Getting started with xUnit.net v3](#)
7. [Docker, Docker Compose documentation](#)
8. [GitHub, Building and testing .NET](#)

Final evidence gate before PDF submission

- Add the public GitHub URL to the cover and verify it while signed out.
- Replace the seven remaining screenshot evidence slots with actual emulator captures.
- Insert the coverage-report screenshot showing the verified 86.4% line rate.
- Insert a screenshot of one green GitHub Actions run with both jobs visible.
- Confirm the public repository contains source, tests, workflow, README, and genuine commit history.
- Export to PDF and verify it remains under the 20-page limit.
- Rehearse the eight-minute demonstration and explain the four patterns without reading notes.

Current honest status

Source: 70%-targeted and presentation-readable. Tests: 65/65 passing with 86.4% line coverage. One real emulator screenshot is included. Remaining: final live workflow check, public GitHub URL/history, green workflow evidence, seven app screenshots, coverage screenshot, and oral defence.